

Project Report

Maksim Petrosyan, Hayk Matshkalyan

CS232: Cybersecurity

Vaagn Toukharian

May 3, 2026

Introduction	3
Threat Model and Problem	3
Background and Research	4
Attack Implementation	5
Detection and Defense System	7
Limitations	8
Future Work	8
Conclusion	8
References	10

Introduction

BadUSB is a type of attack that leverages the reprogrammable firmware in USB devices to carry out malicious actions. It can emulate trusted Human Interface Devices (HID), such as keyboards. Once connected to a computer a BadUSB device can emulate a keyboard and type malicious commands, install backdoors and keyloggers, redirect internet traffic or exfiltrate sensitive data (Roy, 2025).

Because operating systems inherently trust input devices and malware is embedded in USB's firmware, this type of attacks bypass traditional security mechanisms, including antivirus software and user permission controls.

This project investigates the feasibility of BadUSB attacks using a programmable USB device and proposes a behavior-based detection system that identifies malicious activity through keystroke analysis, process monitoring, and event correlation.

Threat Model and Problem

The threat model assumes a Windows-based system, where USB devices and HIDs are inherently trusted by the operating system (Microsoft, n.d). The attacker is assumed to have physical access to the target environment and the ability to introduce a malicious USB device.

In practice, malicious actors often use social engineering techniques such as leaving "lost" reprogrammed USB drives in high-traffic areas (NordVPN, n.d.) or sending the device via mail as a "gift" to victim (Boyd, 2022), which in many cases ends up malicious USB drive being connected to the victim's workstation.

Once connected, the device operates under the same privilege level as the currently logged-in user. Within this model, the attacker can execute arbitrary commands and PowerShell scripts using automated keystrokes. Because these commands appear to the OS as a legitimate user input, they bypass Antivirus and Endpoint Detection and

Response systems. Commands or payloads are usually delivered in seconds, leaving the user with no time to intervene (Roy, 2025).

The core security problem addressed in this project arises from the implicit trust in HID devices by modern operating systems. Since keyboard input is not typically validated or monitored at a behavioral level, malicious automation can bypass traditional endpoint security solutions such as antivirus software and basic intrusion detection systems. As a result, BadUSB-type attacks represent a class of threats that operate below the visibility of conventional security mechanisms.

Therefore, there is a need for detection mechanisms that analyze input behavior and system activity patterns in real time, rather than relying only on signature-based or file-based malware detection approaches

Background and Research

HID devices, such as keyboards, mice and game controllers, are considered trusted input sources by modern operating systems. This implicit trust is necessary to enable plug-and-play functionality without requiring custom drivers and allows for immediate, low-latency communication. However this approach assumes that all HID-originated input is legitimate and operating systems do not perform behavioral validation of input, which introduces a potential security weakness.

BadUSB attacks exploit this design to execute malicious commands (Roy, 2025). First time, the concept of BadUSB was demonstrated during Black Hat USA 2014 (annual cybersecurity conference), where researchers from Security Research Labs (SRLabs) showed that USB device firmware can be reprogrammed to emulate keyboards and inject malicious keystrokes (Nohl & Lell, 2014).

Although our project is inspired by BadUSB research, the implemented prototype is functionally closer to a keystroke injection device, such as USB Rubber Ducky. Unlike

BadUSB attacks, which modify the firmware of existing USB devices, we used a microcontroller which emulates a keyboard.

Traditional endpoint protection mechanisms focus on detecting malicious files or known execution patterns. However, BadUSB attacks bypass these mechanisms as they operate as input events rather than executable files. As a result, most of AV and EDR systems are not able to detect such attacks. Some commercial security solutions have introduced techniques targeting BadUSB attacks. For example, Kaspersky implements a BadUSB Attack Prevention mechanism based on device authorization. When a USB device is connected to the computer and identified as a keyboard by the operating system, the application prompts the user to enter a numerical code generated by the application from this keyboard. Once verified, the system stores device identifiers such as VID/PID and port information, allowing trusted devices to reconnect without repeated authorization (Kaspersky, 2025).

This approach is highly effective against automated keystroke injection attacks, but it relies heavily on user interaction which may reduce usability in environments where keyboards are frequently reconnected.

Attack Implementation

This project implements a keystroke injection attack using a microcontroller-based USB device, in particular, the Digispark ATTiny85, which is a compact, cheap and Arduino-compatible development board. The device is programmed to emulate a Human Interface Device (HID) keyboard and automatically execute a predefined sequence of commands when connected to a target system.

Initially, Digispark drivers were installed in order to enable code uploading from the Arduino IDE to the ATTiny85 microcontroller. The drivers were required for the development phase, so the code can be uploaded to the board. After the firmware is uploaded, the device operates independently as a USB HID. This means it emulates a

standard keyboard on other Windows systems without requiring additional driver installation or user configuration.

```
#include "DigiKeyboard.h"

void setup() {
    DigiKeyboard.delay(8000);

    DigiKeyboard.sendKeyStroke(KEY_R, MOD_GUI_LEFT);
    DigiKeyboard.delay(1000);

    DigiKeyboard.println("cmd");
    DigiKeyboard.delay(1000);

    DigiKeyboard.println("cd %temp%");
    DigiKeyboard.println("echo --- START --- > info.txt");
    DigiKeyboard.println("whoami >> info.txt");
    DigiKeyboard.println("hostname >> info.txt");
    DigiKeyboard.println("ipconfig >> info.txt");
    DigiKeyboard.println("exit");
}

void loop() {}
```

After connecting to a Windows-based workstation, the device waits for 8 seconds to allow the operating system to complete device recognition and bypass detection mechanisms that focus on spawning CMD or PowerShell a few seconds after a USB device is connected. It then opens Windows Run dialog using a keyboard shortcut (WIN+R) and launches command prompt. After opening the command prompt, the device performs basic reconnaissance on the system by executing simple built-in Windows commands to gather initial information about the environment and saves the results to a newly created .txt file in the TEMP folder.

While the implemented payload in this project is limited to collecting data and storing it locally on the victim machine, the same technique can be used to extend the project for more advanced attack scenarios. For example, it can be extended to collect sensitive data on the victim machine and exfiltrate it through the network to external

command-and-control (C2) server. Additionally, the payload can be modified to download and execute malware from a remote server.

Detection and Defense System

Our proposed detection system is a tool that monitors behavior and detects potential BadUSB attacks through real-time analysis of system activity. It does not rely on analysis of signatures of files or known malware patterns, but evaluates multiple signals, such as typing speed, newly created processes and creation of files in TEMP folder.

So the detection logic of our tool can be divided into 3 parts: keystroke behavior analysis, process monitoring, creation of new files in TEMP folder.

The detection tool captures timestamp between keyboard events using input listener, then it calculates the average interval between keystrokes and if it is too low, the behavior is classified as potentially malicious. It is assumed that the human typing speed is slower and less consistent than scripted keystroke injection.

The next part of the detection logic is process creation monitoring. The system continuously monitors newly created processes using system-level process enumeration. For each newly created process, it extracts its name, its parent process name and its command line arguments. Then it scans extracted information against known suspicious command line arguments and suspicious parent - child process relations.

The third part of the detection logic is file creation monitoring. One of the techniques frequently used by attackers is creating files in TEMP folder, so the system detects suspicious files, such as .txt, .ps1, .bat, .vbs, created in TEMP folder.

However, the above mentioned detection rules can create big number of false positive alerts. So in order to reduce the number of false positives, the system does not trigger an alert based only on the detection rules. Instead, it stores a cumulative score of the behavior severity, that is each event has its severity level and the alert is triggered only when cumulative score is higher than predefined minimum.

Limitations

While our system can detect BadUSB attacks, it has several limitations. The system relies on behavioral indicators such as keystroke speed, parent - child process relations and command-line content extracted at runtime. As a result, the detection can be bypassed by introducing randomized delays between keystrokes, or distributing the malicious actions over longer time intervals.

Future Work

USB Insertion Detection: At current stage, the system does not detect USB device insertion, but focuses on detection suspicious behavior. Future improvements can include USB device insertion detection to correlate with further activities.

Integration with System Telemetry and Sysmon: Currently, our implementation relies on runtime process inspection using system APIs. To improve the project, Windows Event Logs or Sysmon can be integrated to enable more reliable process creation logging and full command-line capture.

Conclusion

Overall, our project aimed to show the security risks of BadUSB attacks, which exploits the Windows feature of implicitly trusting HID devices. Using a microcontroller, we created a keystroke injection device to show how automated input can execute commands without traditional malware execution or user awareness.

Then, to show protection mechanisms against such attacks, we implemented a tool that monitors keystroke patterns, process activity and file creation in real time and by correlating such events the system can distinguish between normal user activity and malicious activity.

Overall, the project demonstrates that behavioral analysis is a practical approach for detecting USB-based input injection attacks and provides a foundation for more advanced endpoint security solutions.

References

- Roy, S. (2025). *What Is a BadUSB? Understand the Threat and How to Prevent It*. Ivanti. Retrieved May 3, 2025, from <https://www.ivanti.com/blog/what-is-badusb>
- (n.d.). *Develop Windows device drivers for Human Interface Devices (HID)*. Microsoft. Retrieved May 3, 2025, from <https://learn.microsoft.com/en-us/windows-hardware/drivers/hid/#:~:text=Learn%20to%20develop%20Windows%20device,protocols%20for%20mice%20and%20keyboards.>
- (n.d.). *USB drop attack*. NordVPN. Retrieved May 3, 2025, from <https://nordvpn.com/ru/cybersecurity/glossary/usb-drop-attack/?srslid=AfmBOopBg8NnJQKyHMCpkECyoGBF-Ke2wvHeEoE9yTtRvX7IGFNmgb0D>
- Boyd, C. (2022, January 11). *Attackers are mailing USB sticks to drop ransomware on victims' computers*. Malwarebytes. Retrieved May 3, 2025, from <https://www.malwarebytes.com/blog/news/2022/01/attackers-are-mailing-usb-sticks-to-drop-ransomware-on-victims-computers#:~:text=It's%20quite%20another%20to%20be,notorious%20colonial%20pipeline%20ransomware%20incident.>
- Nohl, K., & Lell, J. (2014). *BadUSB - On Accessories that Turn Evil* by Karsten Nohl + Jakob Lell. Black Hats USA. Retrieved May 3, 2025, from <https://blackhat.com/us-14/video/badusb-on-accessories-that-turn-evil.html>
- Kaspersky (2025, February 26). *BadUSB Attack Prevention*. Retrieved May 3, 2025, from <https://support.kaspersky.com/kes-for-windows/12.3/176739>